

# MODULE 2

## System & Software Security

*Complete Exam-Ready Study Notes (All PDFs Combined)*

---

### PART 0: System Fundamentals

#### System, Security Perimeter & Trust

A System comprises the computing and communication environment over which developers have some control.

#### System Components

- **Security-relevant components:** Crucial components where malfunction or penetration leads to security violations. Examples: OS, computer hardware.
- **Other components:** Objects the system controls and protects. Examples: Programs (not processes), data, terminals, modems.

**Security Perimeter** = the line of demarcation between security-relevant and other components. Think of it as the boundary wall separating critical infrastructure from what it protects.

#### User & Trust

- A User is a person whose information the system protects and whose access is controlled by the system.
- User is trusted with some confidential information.
- System security needs trust in security-related components inside the security perimeter.
- **Trust is built using techniques of identification and authentication.**

#### Why Software Matters for Security

- Virtually ALL information security is implemented in software.
- If software is subject to attack, security is broken — regardless of strength of crypto, access control, or protocols.
- Paul Kocher: **"Complexity is the enemy of security."**

#### Bad Software Examples (Real-World)

- **NASA Mars Lander (\$165 million)** — Crashed into Mars due to error converting English and metric units.
- **Denver Airport** — Buggy baggage system delayed opening 11 months. Cost exceeded \$1 million/day.
- **MV-22 Osprey** — Advanced military aircraft. Lives lost due to faulty software.

## Lines of Code (LOC) & Bug Statistics

| System        | Lines of Code |
|---------------|---------------|
| Space Shuttle | 10,000,000    |
| Netscape      | 17,000,000    |
| Linux Kernel  | 1,500,000     |
| Windows XP    | 40,000,000    |
| Boeing 777    | 7,000,000     |

**Bug calculation:** Conservative estimate: 5 bugs per 1000 LOC. Typical computer = 3000 executables of 100K each = ~50 bugs/exe = 150,000 bugs/computer. A 30,000-node network = 4.5 billion bugs. If 10% security-critical and 10% remotely exploitable = 4.5 MILLION critical flaws!

## Error → Fault → Failure Chain

- **Error** = A programming mistake ("to err is human").
- **Fault** = Error leads to incorrect internal state (internal to the program).
- **Failure** = Fault leads to system departing from expected behavior (externally observable).

"Flaw" is the umbrella term for all three.

**Off-by-One Example:** `char array[10]; array[10] = 'B';` — Array has indices 0-9, so index 10 is an error that corrupts adjacent memory.

**Key Insight:** In software engineering, ensure program does what intended. In SECURE software engineering, ensure it does what intended AND NOTHING MORE.

### MINI NOTES — System Fundamentals

System = computing + communication environment. Security-relevant (OS, hardware) vs Other (programs, data) components. Security Perimeter = boundary. Trust via identification + authentication. ALL security in software — bad software = broken security. 5 bugs/1000 LOC. Error (mistake) → Fault (internal) → Failure (external). Secure software = intended behavior AND NOTHING MORE.

# PART 1: Unintentional (Non-Malicious) Programming Flaws

These are accidental software bugs, logic errors, or oversights during design and coding that create security vulnerabilities. They do NOT stem from malicious intent but can lead to severe system instability, data breaches, or arbitrary code execution.

## 1. Buffer Overflow

Buffers are fixed blocks of memory used to store specific amounts of information. Unless the program has built-in instructions to discard excess data, data in memory adjacent to the buffer will be overwritten. Attackers exploit buffer overflows to corrupt software. Despite being well-understood, they remain a serious security issue.

### Memory Organization (Draw This Diagram in Exam)

Process memory is organized as:

- **Text** — Program code (instructions)
- **Data** — Static variables
- **Heap** — Dynamic data (grows DOWNWARD)
- **Stack** — Local variables, function parameters, RETURN ADDRESS (grows UPWARD)

Text is at low address, Stack is at high address. The Stack Pointer (SP) tracks the current top of the stack.

### How Stack-Based Buffer Overflow Works

The stack has a fixed size. The developer sets aside space. If user input is longer than the designated space and software doesn't verify input length, the stack overflows.

#### Normal Stack Frame (when func(1,2) is called):

b (parameter) | a (parameter) | ret (return address) | buffer[10]

The return address (ret) tells the program WHERE to go after the function finishes. This is the attacker's target.

#### Attack Steps:

1. Attacker sends input longer than buffer size.
2. Extra data overflows past buffer into adjacent memory.
3. Overflow overwrites the return address (ret).
4. When function returns, it jumps to the attacker's chosen address.
5. Attacker injects evil code (shellcode) and makes ret point to it.

6. Program executes attacker's code with program's privileges!

## NOP Sled / Landing Pad

Problem: Attacker may not know exact address of evil code or exact location of ret on stack.

**Solution 1: NOP Landing Pad** — Precede evil code with a long sequence of NOP (No Operation) instructions. Even if ret doesn't land exactly on evil code, it lands on a NOP, slides down to evil code.

**Solution 2: Multiple ret values** — Insert many copies of the new return address to increase chance of hitting the right spot.

## Simple Authentication Bypass via Overflow

If a boolean authentication flag sits in memory right after a buffer, overflowing the buffer can overwrite the flag — allowing anyone to authenticate without knowing the password!

## Heap Overflow Attack

Targets data in the open memory pool known as the heap (dynamically allocated memory). Can corrupt data structures, function pointers, or metadata.

## Integer Overflow Attack

An arithmetic operation results in an integer too large for the integer type meant to store it. Example: A 16-bit unsigned integer holds 0-65,535. If calculation gives 65,536, it wraps to 0. If used for buffer allocation, you get a tiny buffer for large data → buffer overflow.

## Stack Smashing Prevention

| Method                               | Description                                                                                       | Limitation                           |
|--------------------------------------|---------------------------------------------------------------------------------------------------|--------------------------------------|
| <b>NX Bit (Non-executable stack)</b> | Mark stack memory as non-executable                                                               | Some real code runs on stack (Java)  |
| <b>Safe Languages</b>                | Use Java, C# with built-in bounds checking                                                        | Can't use for all systems            |
| <b>Safer C Functions</b>             | strncpy instead of strcpy, snprintf instead of sprintf                                            | Requires developer discipline        |
| <b>Canary / Stack Guard</b>          | Known value placed between buffer and ret. Checked before return. If changed = overflow detected. | Attacker may target handler (MS /GS) |

**Canary Details:** Value 0x000aff0d contains NULL, newline, EOF, CR — common string terminators that make it hard for attacker to include in overflow input. Microsoft's /GS flag uses 'security cookie' but the user-supplied handler can itself be attacked.

### MINI NOTES — Buffer Overflow

Buffer = fixed memory. Overflow = extra data overwrites adjacent memory. Stack: Text→Data→Heap(↓)→Stack(↑). Stack stores local vars + params + RETURN ADDRESS. Attack: overflow buffer → overwrite ret → redirect to evil code. NOP sled = NOPs before evil code (don't need exact address). Prevention: NX bit, safe languages (Java/C#), safer functions (strncpy), canary (0x000aff0d between buffer & ret). Heap overflow = dynamic memory. Integer overflow = wraps around → tiny buffer for large data.

## 2. Incomplete Mediation

Incomplete mediation refers to the failure of a system to enforce security policies consistently across all entry points. It involves verifying that a subject is authorized to perform an operation on an object, but failing to do so completely.

### Classic URL Manipulation Example:

Valid: `http://site.com/buy?item=1&price=100`

Attack: `http://site.com/buy?item=1&price=1`

If the server doesn't re-validate the price (trusts client-side validation), the attacker buys a ₹100 item for ₹1!

**Key Point:** Input validated on client but NOT on server = incomplete mediation. NEVER trust client-side data. Even the Linux kernel (open-source, written by experts) has many buffer overflows due to incomplete mediation.

### Examples

- **Input Validation:** Web app doesn't validate user input on ALL pages → attacker submits malicious data the system accepts.
- **Session Management:** Session not properly validated after login → attacker hijacks the session.

### Prevention Strategies

- **Consistent Input Validation** — Validate ALL user inputs at EVERY entry point.
- **Access Controls** — Use a reference monitor for complete mediation. Verify user permissions before allowing actions.
- **Regular Security Audits** — Module testing, integration testing to ensure all parts function correctly and securely.

### MINI NOTES — Incomplete Mediation

Failure to enforce security at ALL entry points. URL example: price=100 changed to price=1 (server doesn't recheck). strcpy(buffer, argv[1]) without length check = incomplete mediation. NEVER trust client-side validation. Prevention: validate at EVERY entry point, reference monitor, regular audits.

### 3. Time-of-Check to Time-of-Use (TOCTOU)

TOCTOU is a race condition security vulnerability occurring when a system verifies a condition (e.g., file permissions) but the resource changes before the system uses it. An attacker manipulates the resource during this gap, leading to unauthorized access, privilege escalation, or data corruption.

**The Vulnerability Window:** A small, critical time interval between the CHECK and the USE.

#### Classic File System Example (Memorize This)

1. **CHECK:** A privileged (setuid) program checks if /home/user/log is owned by the user (safe to write).
2. **RACE:** Attacker quickly deletes /home/user/log and creates a symbolic link: `ln -s /etc/passwd /home/user/log`
3. **USE:** Program opens /home/user/log, but now it's writing to /etc/passwd!

#### mkdir Race Condition

1. **Step 1:** mkdir allocates space for new directory.
2. **Step 2 (ATTACKER):** Creates a link to the password file (between allocation and ownership transfer).
3. **Step 3:** mkdir transfers ownership to user. User now OWNS the password file!

**Key Point:** Race conditions may be MORE prevalent than buffer overflows but HARDER to exploit. Buffer overflow is 'low hanging fruit'.

**Mitigation:** Make operations ATOMIC (combine check + use into one indivisible operation). Use file descriptors instead of file names.

#### MINI NOTES — TOCTOU

Race condition: CHECK a condition → attacker CHANGES resource → system USES changed resource. Symlink example: check /home/user/log → attacker replaces with link to /etc/passwd → program writes to /etc/passwd. mkdir: allocate → attacker links to password file → transfer ownership. Fix: ATOMIC operations or file descriptors. More prevalent than buffer overflow but harder to exploit.

### 4. Undocumented Access Points

Often called 'rogue access points' or 'hidden interfaces' — network devices or software functionalities operating without authorization or public knowledge, posing significant security risks. They bypass security controls, facilitate lateral movement, or establish persistence.

- **Rogue Access Points:** Unauthorized Wi-Fi APs on a secure network. Create an 'open door' bypassing perimeter defenses.
- **Hidden Features / Backdoors ('chicken bits'):** Manufacturer's testing/maintenance features that can be exploited to read/write memory, execute arbitrary code.

## 5. Off-by-One Error

A programming error where a loop iterates one time too many or too few, or an array is accessed at index  $n$  when valid indices are 0 to  $n-1$ .

**Example:** `char array[10]; array[10] = 'B';` — Valid indices are 0-9, so writing to index 10 overwrites adjacent memory. This is both an off-by-one error AND can cause a buffer overflow.

## 6. Integer Overflow

An arithmetic operation produces a result too large (or too small) for the integer type. The value wraps around.

**Example:** 16-bit unsigned max = 65,535. If result = 65,536, wraps to 0. If used for buffer allocation → tiny buffer allocated for large data → buffer overflow.

## 7. Unterminated Null Terminated String

A character array intended to be a null-terminated string lacks the `\0` character at its end. In C/C++, functions like `strlen()`, `printf()`, `strcpy()` read until they find `\0`. If missing, they read past the buffer.

### Consequences

- **Buffer Overflows** — Function writes past the buffer end.
- **Security Vulnerabilities** — Common source of security problems.
- **Crashes** — Segmentation fault or access violation.
- **Logic Errors** — Outputs garbage data from adjacent memory.

## 8. Parameter Length, Type, and Number

Enforcing strict min/max byte limits on input data (URL parameters, form fields) to prevent buffer overflows and SQL injection.

- **Buffer Overflow Protection:** Excessively long parameters exploit weak handling → crashes or code execution.

- **Input Validation:** WAFs check input lengths against defined limits.
- **SPI (Security Parameter Index):** In IPsec, a 32-bit identifier linking packets to specific security associations.
- **Password Policies:** Define minimum character lengths and complexity.

## 9. Unsafe Utility Programs

- **Trojan Horse Utilities:** Fake antivirus/system cleaners that steal data.
- **Rogue Registry Cleaners:** Claim hundreds of 'errors' to scare users into buying; may cause system instability.
- **Bundled Adware/Spyware:** Free utilities that install unwanted software.

## 10. Race Condition

Security processes should be atomic — occur 'all at once'. Race conditions arise when security-critical processes occur in stages and an attacker makes changes between stages, often between authorization and ownership transfer.

**Prevention:** Make security-critical processes atomic. Use proper synchronization (locks, mutexes, semaphores). Not always easy in practice.

### MINI NOTES — All Other Flaws

Undocumented Access Points: rogue Wi-Fi APs, hidden 'chicken bits'. Off-by-One: array[10] on size-10 array. Integer Overflow: wraps around → tiny buffer. Unterminated Null String: missing \0 → read past buffer → overflow/crash/garbage. Parameter Length: enforce min/max, WAF validation, SPI in IPsec. Unsafe Utilities: fake tools, rogue cleaners, bundled adware. Race Condition: make processes ATOMIC.



## PART 2: Types of Malware

Malware is software that infects systems without user consent to steal sensitive data (bank details, passwords, personal emails), disrupt operations, or alter core system behavior. It can exfiltrate confidential information, corrupt or delete files, and impair system availability or integrity.

**Fred Cohen's initial virus work in 1980s** used viruses to break MLS (Multi-Level Security) systems.

### Malware Transmission & Propagation

- Setup and installer programs
- Attached files and document viruses
- Autorun
- Appended viruses / viruses that surround a program

### Harm from Malicious Code

**To Users/Systems:** Sending email to contacts, deleting/encrypting files, modifying Windows registry, stealing passwords, attaching to critical files, hiding copies in multiple locations.

**To the World:** Malware can infect millions at geometric rate. Infected systems become staging areas for new infections.

## 1. Viruses

A virus is a malicious executable code attached to another executable file. It spreads when an infected file is passed from system to system. Cannot run independently — needs a host program. Once a program virus is active, it infects other programs on the computer.

### Virus Pseudocode (From Your PDF)

```
infect(); if trigger() then payload();
```

### Three Components

**1. Infection Mechanism:** Locates target files (.exe, .doc, boot sector), inserts malicious code, ensures virus runs when host runs. Purpose: replication and spreading.

**2. Payload:** The harmful action — deletes files, encrypts data, steals passwords, opens backdoors. Some viruses have NO destructive payload (only spread). Some mild (display message). Some severe (ransomware behavior).

**3. Trigger (Activation Mechanism):** Controls WHEN payload executes. Types: specific date, number of infections reached, system reboot, user login, file execution, specific program installed.

**Flow:** Infection → Dormant Stage → Trigger Condition → Payload Execution

### Where Do Viruses Live?

- **Boot sector** — Take control before anything else
- **Memory resident** — Stays in memory
- Applications, macros, data
- **Library routines, compilers, debuggers, even virus checkers!** — Particularly nasty

### Virus Classification by TARGET

#### A) Boot Sector Virus:

- **Primary boot infection:** Infects MBR, activated at startup, spreads via bootable media.
- **Secondary boot infection:** Infects additional boot sectors or removable drives.
- Characteristics: Executes at startup, difficult to detect, can prevent booting. Example: Michelangelo virus.

#### B) Executable File Infectors:

- **Prepending virus:** Code placed at BEGINNING of file. Runs before original, transfers control back. File size increases.
- **Appending virus:** Code placed at END of file. Modifies entry point. MOST COMMON type.
- **Overwriting virus:** REPLACES original code. Destroys program. File size may NOT change (harder to detect by size). High damage — only fix: delete + reinstall. Example: Trivial.88.D

#### C) Companion Virus:

Does NOT modify original file. Creates new malicious file with same name but different extension. Exploits OS execution order (DOS: .com before .exe). Example: editor.exe exists → virus creates editor.com → user runs 'editor' → OS runs .com first → virus executes → may launch editor.exe → user suspects nothing.

#### D) Macro Virus (Data File Infectors):

A macro is a small program inside documents (Word, Excel). Embedded in .doc, .xls files. Executes when document opened. Platform independent. Spreads via email attachments. Example: Melissa Virus.

## Virus Classification by CONCEALMENT

| Type                | How It Hides                                                                        | Detection Difficulty                                          |
|---------------------|-------------------------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Encrypted</b>    | Decryptor + Encrypted body. Decryptor decrypts in memory, then virus runs.          | Decryptor is CONSTANT → antivirus detects decryptor signature |
| <b>Oligomorphic</b> | Pool of decryptors (not just one). Varying keys.                                    | Marginal difficulty — check loop variants                     |
| <b>Polymorphic</b>  | Extremely large number of decryptor loops. Mutation engine changes loop every time. | Much harder than oligomorphic                                 |
| <b>Metamorphic</b>  | NO decryption loops. Entire virus code rewrites itself every infection.             | HARDEST to detect — no constant signature                     |

**Progression:** Encrypted → Oligomorphic → Polymorphic → Metamorphic (hardest)

## Virus Writing Methods (Code Obfuscation)

- **Instruction Equivalence:**  $X=0 \rightarrow X=10; X=X-10$
- **Instruction Sequence Equivalence:**  $total=a+b \rightarrow total=a; total+=b$
- **Instruction Reordering:**  $a=5; b=10; c=a+b \rightarrow b=10; a=5; c=a+b$
- **Register/Variable Renaming:**  $count=10 \rightarrow a9f2=10$
- **Convoluting Programs:**  $if\ x==10 \rightarrow if\ (x*5-40)==10$
- **Inlining & Outlining:** Expand function calls inline or move inline code into functions.

### MINI NOTES — Viruses

Virus = malicious code ATTACHED to executable. Needs host. Pseudocode: `infect(); if trigger() then payload()`. 3 parts: Infection mechanism (spread), Payload (damage), Trigger (timing). Lives in: boot sector, memory, apps, libraries, even virus checkers. By Target: Boot sector (primary MBR/secondary), Executable (prepend/append MOST COMMON/overwrite), Companion (same name, different extension, .com before .exe), Macro (inside docs, platform independent, Melissa). By Concealment: Encrypted→Oligomorphic→Polymorphic→Metamorphic. 6 writing methods for obfuscation.

## 2. Worms

Worms replicate themselves on the system, attaching to different files and looking for pathways between computers (network shares). Worms usually slow down networks. Unlike viruses, worms can run by themselves — no host needed. After affecting a host, spreads very quickly.

- **IM Worms:** Spread through instant messaging platforms.

- **Email Worms:** Self-replicating malware spreading through email. Does not need to attach to existing program. Examples: Melissa (1999), ILOVEYOU (2000), MyDoom (2004).

**Key difference:** Virus needs host. Worm runs independently. Worm = active propagation. Virus = passive propagation.

### 3. Trojan Horse

Malware that carries out malicious operations under the appearance of a desired operation (e.g., playing a game). Binds to NON-executable files (images, audio) — unlike viruses which bind to executables. Does NOT self-replicate.

**Example (from PDF):** freeMusic.mp3 looks like an audio file but is actually an application. Double-click → iTunes opens + 'Wild Laugh' plays + message box appears. Could have done ANYTHING the user can do — delete files, download files, launch apps.

### 4. Ransomware

Ransomware grasps a computer system or data until victim makes payment. Encrypts data with a key unknown to user. User must pay ransom (usually cryptocurrency) to retrieve data. No guarantee of key after payment.

### 5. Adware & Spyware

**Adware:** Displays unwanted ads/pop-ups. Comes bundled with software downloads. Generates revenue via ads. Not self-replicating. Objective is marketing.

**Spyware:** Steals private information for a third party. Collects passwords, credit card numbers, software license keys. Does not replicate. Different form of Trojans. Often downloaded via 'drive-by download' when viewing webpages.

### 6. Logic Bombs

A malicious program with a trigger to activate malicious code. Remains non-functioning until trigger event. Has two parts: Payload (malicious code) and Trigger (boolean logic). Time bombs are examples of logic bombs.

Can destroy HARDWARE — cooling fans, hard drives, power supplies — by overdriving them until overheat/fail.

### 7. Rootkits & Backdoors

**Rootkit:** Modifies the OS to create a backdoor. Attackers use the backdoor for remote access. Exploits software vulnerabilities to modify system files.

**Backdoor:** Bypasses usual authentication. Grants future access even after original vulnerability is fixed.

## 8. Keyloggers

Records everything the user types to obtain passwords and sensitive information, sends them to the attacker.

| Malware           | Needs Host? | Self-Replicates?  | Key Feature                    |
|-------------------|-------------|-------------------|--------------------------------|
| <b>Virus</b>      | YES         | Yes (via host)    | Attaches to executables        |
| <b>Worm</b>       | NO          | Yes (independent) | Spreads via network, slows it  |
| <b>Trojan</b>     | NO          | NO                | Disguised as useful program    |
| <b>Ransomware</b> | NO          | Varies            | Encrypts data, demands payment |
| <b>Spyware</b>    | NO          | NO                | Steals info secretly           |
| <b>Adware</b>     | NO          | NO                | Shows unwanted ads             |
| <b>Logic Bomb</b> | Embedded    | NO                | Trigger-based activation       |
| <b>Rootkit</b>    | NO          | NO                | Modifies OS for backdoor       |
| <b>Keylogger</b>  | NO          | NO                | Records all keystrokes         |

### MINI NOTES — All Malware Types

Virus (needs host, passive propagation), Worm (independent, active propagation, slows network), Trojan (disguised, non-executable files), Ransomware (encrypts + demands payment), Adware (unwanted ads), Spyware (steals info, drive-by download), Logic Bomb (trigger-based, can destroy hardware), Rootkit (modifies OS → backdoor), Backdoor (bypasses auth, persists after fix), Keylogger (records keystrokes). Virus = passive, Worm = active propagation.

# Malware Detection Methods

## 1. Signature Detection

A signature is a string of bits (or hash value) found in malware. Search all files for known signatures. Most popular detection method.

| Advantages                       | Disadvantages                                              |
|----------------------------------|------------------------------------------------------------|
| Effective on traditional malware | Signature files can be large (10,000s) → slow scanning     |
| Minimal burden for users/admins  | Must be kept up to date                                    |
|                                  | Cannot detect UNKNOWN viruses                              |
|                                  | Cannot detect new malware types (polymorphic, metamorphic) |

## 2. Change Detection

Viruses must live somewhere. If a file changes, it may be infected. Hash files → store securely → recompute and compare → if changed, possibly infected.

| Advantages                            | Disadvantages                                                             |
|---------------------------------------|---------------------------------------------------------------------------|
| Virtually no false negatives          | Many files change often → many false alarms (false positives)             |
| Can detect previously UNKNOWN malware | Heavy burden on users/admins                                              |
|                                       | If suspicious change detected, then what? May still need signature system |

## 3. Anomaly Detection

Monitor system for anything 'unusual' or 'virus-like'. Must first define 'normal' — and normal can change!

| Advantages                          | Disadvantages                                    |
|-------------------------------------|--------------------------------------------------|
| Chance of detecting unknown malware | Unproven in practice                             |
|                                     | Attacker can make abnormal look normal (go slow) |
|                                     | Must combine with another method                 |
|                                     | As difficult as AI                               |

### MINI NOTES — Malware Detection

3 methods: 1) Signature Detection (most popular, bit strings/hashes, can't detect unknown malware), 2) Change Detection (hash + compare, catches unknown but many false positives), 3) Anomaly

Detection (monitor for unusual, must define 'normal', unproven, combine with others). Best approach: COMBINE methods.

## Anti-Malware Strategies

1. **Proactive Prevention:** Regularly updating software and OS to patch vulnerabilities.
2. **Defense-in-Depth:** Multiple layers — firewalls, IDS, web filtering.
3. **Zero Trust Architecture:** Strict, least-privilege access policy to restrict malware movement.
4. **User Training:** Educating users to avoid suspicious links and email attachments.
5. **Data Backups:** Regular, OFFLINE backups to restore data in case of ransomware.
6. **Network Segmentation:** Dividing networks into smaller, isolated zones to contain breaches.

## Anti-Malware Tools

1. **Antivirus/Anti-Malware:** Signature-based detection + real-time scanning. Examples: Malwarebytes, Microsoft Defender.
2. **EDR (Endpoint Detection & Response):** Advanced monitoring, detecting, responding at endpoint level.
3. **Network Security Tools:** Firewalls + traffic analyzers (Wireshark, Fiddler).
4. **Email & Web Filtering:** Block malicious websites and phishing emails.
5. **Deception Technology:** Honeypots and decoys to lure and analyze attackers.
6. **Memory Protection:** Defend against fileless malware operating in memory.

## PART 3: Countermeasures

### Countermeasures for Users

1. Use software acquired from reliable sources.
2. Test software in an isolated environment.
3. Only open attachments when you know them to be safe.
4. Treat every website as potentially harmful.
5. Create and maintain backups.

### Countermeasures for Developers

#### Software Engineering Techniques

1. **Information Hiding:** Describing what a module does, NOT how (encapsulation).
2. **Modularity:** Coupling (inter-module dependency — should be LOW) and Cohesion (intra-module relatedness — should be HIGH).
3. **Mutual Suspicion:** Each module treats others as potentially untrustworthy.
4. **Confinement:** Possible damage does not spread to other parts of the system.
5. **Simplicity:** Complexity is often the enemy of security.
6. **Genetic Diversity:** Diversity reduces targets susceptible to one attack type.
7. **Testing.**

### Code Testing Types

| Testing Type         | What It Tests                             |
|----------------------|-------------------------------------------|
| Unit Testing         | Individual components                     |
| Integration Testing  | Modules working together                  |
| Function Testing     | Features work as expected                 |
| Performance Testing  | Speed, stability under load               |
| Acceptance Testing   | Meets user requirements                   |
| Installation Testing | Works in target environment               |
| Regression Testing   | New changes don't break existing features |
| Penetration Testing  | Simulated attacks to find vulnerabilities |

### Additional Testing (From Mark Stamp PDF)



- **Active Fault Detection:** Don't wait for system to fail — actively try to make it fail (attackers will!).
- **Fault Injection:** Insert faults into the process, even if no obvious way for such a fault to occur.
- **Bug Injection:** Insert known bugs into code. See how many are found. Use this to ESTIMATE total number of bugs. Assumes injected bugs similar to unknown bugs.

## Security Design Principles (8 Principles)

| Principle                         | Description                                                             | Breach Example                                   |
|-----------------------------------|-------------------------------------------------------------------------|--------------------------------------------------|
| <b>1. Least Privilege</b>         | Grant MINIMUM access required. Limits damage from compromised accounts. | Target 2013 — Vendor access exploited            |
| <b>2. Economy of Mechanism</b>    | Keep security mechanisms SIMPLE. Smaller code = fewer vulnerabilities.  | Heartbleed 2014 — OpenSSL complexity             |
| <b>3. Open Design</b>             | Security should NOT rely on secrecy. Public review strengthens systems. | Weak proprietary IoT encryption                  |
| <b>4. Complete Mediation</b>      | Check EVERY access request. Prevents bypass attacks.                    | Equifax 2017 — Unpatched vulnerability           |
| <b>5. Permission Based</b>        | Default should be DENIAL. Access only when explicitly permitted.        |                                                  |
| <b>6. Separation of Privilege</b> | Require MULTIPLE conditions for critical operations.                    | Twitter 2020 — Social engineering on admin tools |
| <b>7. Least Common Mechanism</b>  | Minimize SHARED resources. Reduces cross-user attack risk.              | Cloud multi-tenant data exposure                 |
| <b>8. Ease of Use</b>             | Security must be user-friendly. Complex systems cause user bypass.      | Weak passwords from difficult policies           |

## Additional Security Countermeasures

- **Penetration Testing for Security**
- **Proofs of Program Correctness** — Mathematical proofs that code behaves as specified.
- **Validation** — Verifying software meets requirements.
- **Defensive Programming** — Anticipate what could go wrong, not just write correct code.
- **Trustworthy Computing Initiative** — All developers undergo security training + secure practices.

## Countermeasures That DON'T Work

**1. Penetrate-and-Patch:** Fails because it is hurried, misses the context of the fault, and focuses on one failure, not the complete system. You fix one hole without understanding WHY it existed — similar holes exist elsewhere.

**2. Security by Obscurity:** The belief that a system can be secure as long as nobody outside knows its internal mechanisms. Fails because once the secret is discovered, there is NO security left. True security should work even if design is known (Kerckhoffs' principle).

#### **MINI NOTES — Countermeasures**

Users: reliable sources, isolated testing, safe attachments, websites dangerous, backups. Developers: information hiding, modularity (low coupling/high cohesion), mutual suspicion, confinement, simplicity, genetic diversity, testing. 8 Testing types. Active fault detection + fault injection + bug injection. 8 Design Principles: Least Privilege, Economy of Mechanism, Open Design, Complete Mediation, Permission Based, Separation of Privilege, Least Common Mechanism, Ease of Use. DON'T WORK: Penetrate-and-Patch (hurried, misses context) + Security by Obscurity (once known = zero security).

## PART 4: Secure Software Development Lifecycle (SSDLC)

SSDLC is an extension of traditional SDLC where security is integrated into EVERY phase of development — not added at the end. Goal: move away from 'penetrate and patch'. Requires more time and effort but produces fewer and less severe flaws.

### SSDLC Principles

1. **Security by Design:** Think about security from the START. Include security requirements in initial planning.
2. **Continuous Monitoring:** Ongoing process at EVERY step. Not one-time but continuous.
3. **Risk Assessment:** Evaluate potential risks EARLY. Identify vulnerabilities, prioritize threats.
4. **Education and Training:** Everyone involved knows about security. Developers have skills to handle concerns.
5. **Collaboration:** Teamwork between developers, operations, and security teams.

### Phases of SSDLC

1. **Planning:** Identify security requirements, possible risks, create security plan.
2. **Design:** Build security features into architecture. Principles: Favor Simplicity, Trust with Reluctance, Defend in Depth.
3. **Implementation:** Secure coding practices + code reviews. Static Analysis tools: SonarQube (20%), FindBugs (19%), Checkstyle (16%), PMD (12%). Also: Symbolic Execution.
4. **Testing:** Penetration testing, fuzz testing (random/malformed data), employ attack patterns.
5. **Deployment:** Secure release process, precautions during this stage.
6. **Maintenance:** Continuous monitoring, regular updates for new threats.

### Secure Development Aspects (Mark Stamp)

- **Design:** Avoid high-level errors (costly/impossible to fix later). Verify assumptions. Formal methods possible but only for simple cases.
- **Hazard Analysis / Threat Modeling:** Hazard list, 'what ifs', Schneier's attack tree. Formal: HAZOP, FMEA, FTA.
- **Peer Review:** 3 levels — Review (informal), Walk-through (semi-formal), Inspection (formal). Very effective.
- **Configuration Management + Postmortem for Mistakes.**

## SDLC vs SSDLC Comparison

| Aspect               | SDLC                           | SSDLC                                        |
|----------------------|--------------------------------|----------------------------------------------|
| <b>Focus</b>         | Delivering functional software | Security throughout the process              |
| <b>Objective</b>     | Business requirements          | Integrating security measures                |
| <b>Security</b>      | Mainly during testing phase    | At EVERY stage                               |
| <b>Testing</b>       | Security testing is separate   | Integrated into EACH phase                   |
| <b>Documentation</b> | Functionality and features     | Threat models, risk assessments              |
| <b>Speed</b>         | Timely delivery                | Slightly longer (security overhead)          |
| <b>Adaptability</b>  | More adaptable                 | More effort for changes (stringent security) |

## Methodologies

- **BSIMM** — Building Security In Maturity Model
- **Microsoft SDL** — Microsoft Security Development Lifecycle
- **OpenSAMM** — Software Assurance Maturity Model

## Continuous Security (DevOps)

Old School: Plan → Code → Test → Deploy → Monitor (months/years).

New School (DevOps): Plan → Assemble → Code → Test → Deploy → Monitor (hours/days).  
Uses GitHub, CI/CD, Docker, AWS.

**Key:** Continuous delivery needs continuous security. Security automation. Integrate into CD tools (GitHub, Bitbucket, Travis CI, Jenkins). Audit third-party components.

### MINI NOTES — SSDLC

SSDLC = security at EVERY phase (not just testing). 5 Principles: Security by Design, Continuous Monitoring, Risk Assessment, Education, Collaboration. 6 Phases: Planning → Design → Implementation → Testing → Deployment → Maintenance. Design: Favor Simplicity, Trust with Reluctance, Defend in Depth. Implementation: Static Analysis (SonarQube/FindBugs) + Symbolic Execution. Testing: Pen testing + Fuzz testing. Hazard analysis: HAZOP, FMEA, FTA, attack trees. Peer review: Review/Walk-through/Inspection. Methodologies: BSIMM, MS SDL, OpenSAMM. DevOps = continuous delivery needs continuous security.

## PART 5: Privacy by Design (PbD)

Privacy by Design is an approach developed by Ann Cavoukian (former Information & Privacy Commissioner of Ontario, Canada) that requires privacy to be embedded right from the design stage throughout the development lifecycle — privacy becomes integral to core functionality, not an afterthought.

**Core Idea:** "Privacy should be proactive, embedded into design, and automatically protected throughout the entire lifecycle of data."

### 7 Foundational Principles of PbD

#### Principle 1: Proactive not Reactive; Preventative not Remedial

A privacy-first attitude. Instead of reacting to privacy risks when they happen, actively build processes to PREVENT them from occurring.

#### Principle 2: Privacy as the Default Setting

Users shouldn't have to worry about privacy settings. Highest protection automatically applied. Includes:

- **Collection limitation** — Only collect what you're legally allowed to.
- **Data minimization** — Collect absolute minimum necessary.
- **Use, retention & disclosure limitation** — Don't use for other purposes, don't keep longer than needed, don't disclose unless necessary.
- **Security** — Encryption for confidentiality, integrity, availability.

#### Principle 3: Privacy Embedded into Design

Can't be tacked on at the end. Every decision filtered through privacy-first mindset promoting both functionality AND privacy.

#### Principle 4: Full Functionality — Positive-Sum, not Zero-Sum

No false trade-offs. Privacy AND functionality can coexist (win-win). Zero-sum = trade-offs. Positive-sum = integrate seamlessly.

#### Principle 5: End-to-End Security — Full Lifecycle Protection

From data collection to destruction — everything in between secured. Data remains confidential, true to original form, and accessible.

#### Principle 6: Visibility and Transparency — Keep it Open

Openness builds accountability and trust. Document and communicate clearly. Accessible complaint process + independent verification.

### **Principle 7: Respect for User Privacy — Keep it User-Centric**

Users' privacy interests always in mind. Put power in user's hands to manage their own data. Best UX puts privacy first.

#### **MINI NOTES — Privacy by Design**

PbD by Ann Cavoukian. Privacy embedded from design, not afterthought. 7 Principles: 1) Proactive not Reactive (prevent), 2) Privacy as Default (auto highest protection — collection limitation, data minimization, use/retention/disclosure limitation, security/encryption), 3) Embedded into Design (not tacked on), 4) Positive-Sum (privacy + functionality, no trade-offs), 5) End-to-End Security (collection to destruction), 6) Visibility & Transparency (open, accountable, independent verification), 7) Respect for User Privacy (user-centric, users control own data).

## PART 6: IDS/IPS & Firewalls

### Intrusion Detection & Prevention

**IDS** = Process of monitoring events in a system/network.

**IPS** = Process of detecting intrusion signs and attempting to STOP them.

**IDPS** = Collective system combining IDS & IPS.

### Types of Intruders

- **Masquerader:** Outsider pretending to be a legitimate user.
- **Misfeasor:** Legitimate user who MISUSES their privileges.
- **Clandestine:** Intruder who avoids audit and detection systems entirely.

### IDPS Technologies

- **Network-based:** Monitors network segment, network & application protocols.
- **Wireless:** Monitors wireless traffic and protocols.
- **Network Behavior Analysis:** Detects unusual traffic flows, DDoS, malware, policy violations.
- **Host-based:** Monitors individual host systems.

### IDS Detection Techniques

| Technique                         | How It Works                                             | Key Limitation                                                    |
|-----------------------------------|----------------------------------------------------------|-------------------------------------------------------------------|
| <b>Signature-based</b>            | String matching against known attack signatures          | Cannot detect unknown threats; no memory of previous requests     |
| <b>Anomaly-based</b>              | Compares observed vs normal profiles (static or dynamic) | False positives; dynamic profiles fooled by evasion               |
| <b>Stateful Protocol Analysis</b> | Compares against benign protocol behavior profiles       | Resource intensive; misses attacks within valid protocol behavior |

### Firewalls

A single point of defense between two networks. Can be a router or group of routers for packet filtering plus application-level proxy services. Mechanisms: Allow or Block.

**DMZ (Demilitarized Zone):** Separates external network from internal. Firewalls placed between internet & DMZ, and DMZ & internal network.

| Type                     | Layer               | Speed   | Key Feature                                         | Main Limitation                                 |
|--------------------------|---------------------|---------|-----------------------------------------------------|-------------------------------------------------|
| <b>Packet Filtering</b>  | Transport           | Fastest | Simple rules, NAT, no client config                 | Stateless, no app layer understanding, no audit |
| <b>Circuit Level</b>     | Session + Transport | Medium  | Validates TCP/UDP sessions, anti-spoofing           | Only TCP, limited audit, no higher-layer checks |
| <b>Application Layer</b> | Application         | Slowest | Full protocol understanding, proxy, excellent audit | Slow, not scalable, requires client changes     |

### MINI NOTES — IDS & Firewalls

IDS (monitor), IPS (detect+stop), IDPS (combined). Intruders: Masquerader (outsider), Misfeasor (insider misuse), Clandestine (avoids detection). Technologies: Network, Wireless, Network Behavior, Host-based. Detection: Signature (known attacks), Anomaly (vs normal profile), Stateful Protocol (vs benign protocol). Firewalls: Packet Filtering (transport, fastest, stateless), Circuit Level (session, validates sessions, anti-spoof), Application Layer (app layer, proxy, slowest but strongest, best audit). DMZ separates external from internal.



## PART 7: Miscellaneous Attacks

### Salami Attack

Programmer 'slices off' tiny amounts of money that are hard for victims to detect. Over time, accumulates significant wealth.

- **Bank:** Slices fractions of a cent from interest calculations into own account. No customer notices.
- **Tax:** Added cents to employees' tax withholding, credited to his own tax → big refund.
- **Taco Bell:** \$2.99 item registered as \$0.01 → pocketed \$2.98 each time.
- **Gas pump:** Chip overstated gas pumped. Programmed to be accurate at 5 and 10 gallons (inspector test amounts)!

### Linearization Attack

Program checks serial number one character at a time. Correct character takes slightly longer. Attacker tries all characters, finds which takes most time = correct one. Repeats for each position.

**Math:** 8-char serial, 128 values each. Brute force =  $128^8 = 2^{56}$ . Linearization =  $8 \times (128/2) = 2^9 = 512$  tries — TRIVIAL!

**TENEX example:** Passwords checked one char at a time. Page fault when correct character guessed. Page fault register was user-accessible → easy attack.

### Time Bomb

Donald Gene Burleson (1986): Told employer to stop withholding taxes → refused → fired → his planted 'time bomb' deleted important company data. Fined \$11,800 in 1988. One of first computer crime cases.

#### MINI NOTES — Misc Attacks

Salami: slice tiny amounts undetectably (bank interest fractions, tax withholding, cash register).  
Linearization: check serial one char at a time → timing reveals correct chars.  $128^8$  brute force reduced to  $8 \times 64 =$  trivial. TENEX page fault attack. Time Bomb: planted by disgruntled employee, activates after event (e.g., being fired).